

ESCOLA NAVAL
MARINHA PORTUGUESA

INTELIGÊNCIA ARTIFICIAL

Ano letivo 2026–2027

Docente: António Gonçalves



Como encontra o robô um caminho até ao objetivo?

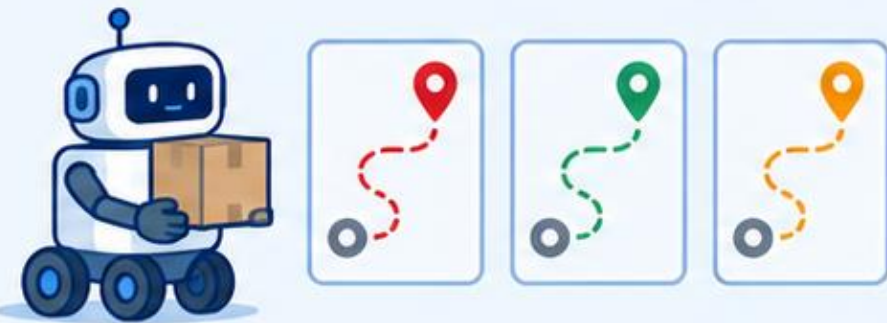
Na primeira semana vimos como o agente decide. Agora vamos perceber como constrói um plano antes de agir.

1 O que já sabemos

O robô tem uma tarefa, observa o ambiente e escolhe ações para atingir um objetivo.

Definimos requisitos e critérios para comparar percursos. Na arquitetura híbrida, o planeamento propõe caminhos até ao destino.

Nos exemplos anteriores, os percursos candidatos já eram apresentados.



2 A pergunta que falta

Como encontra o programa os percursos possíveis?

Conhecer o destino não indica que movimentos devem ser executados.

É necessário usar o mapa e as ações permitidas para construir uma sequência que leve o robô do início ao destino.



3 O que vamos estudar



Representar a situação inicial, as ações e os seus efeitos.



Explorar alternativas com diferentes estratégias de procura.



Verificar se o plano cumpre os requisitos e comparar o seu custo.

A solução procurada é uma sequência de ações.

O mesmo robô, uma nova pergunta



O robô parte de B com 8 unidades de energia e deve chegar a D com uma reserva mínima de 2 unidades.

Que sequência de movimentos permite cumprir a tarefa?



Para começar: mapa e custos conhecidos, movimentos com efeitos previsíveis e ambiente estável durante o planeamento.

A procura permite encontrar planos para um agente orientado para objetivos.

Formulação e procura não informada

O que vamos aprender nesta aula

1

Formular o problema

Estado inicial, ações, transições, objetivo, restrições e custos.



2

Representar a procura

Estados, nós e alternativas por explorar.



3

Aplicar três estratégias

Procura em largura (BFS), em profundidade (DFS) e de custo uniforme (UCS).



4

Comparar os resultados

Caminho encontrado, custo da solução, tempo e memória da procura.



5

Verificar a solução e as garantias

Cumprimento dos requisitos e condições para encontrar uma solução ou uma solução de menor custo.



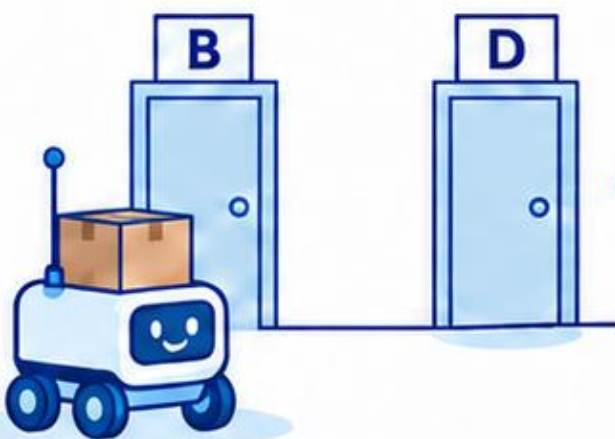
Começamos por transformar a tarefa do robô em requisitos concretos.

Do PEAS aos requisitos do problema

Na primeira semana usámos o PEAS para descrever a tarefa do robô. Agora usamos essa informação para preparar a procura de um plano.

Tarefa e informação inicial

Transportar uma encomenda de B para D.



Neste exemplo, o robô começa em B com **12 unidades** de energia.

S — Sensores

Fornecem informação para estimar a posição e medir a carga disponível.

Condições obrigatórias

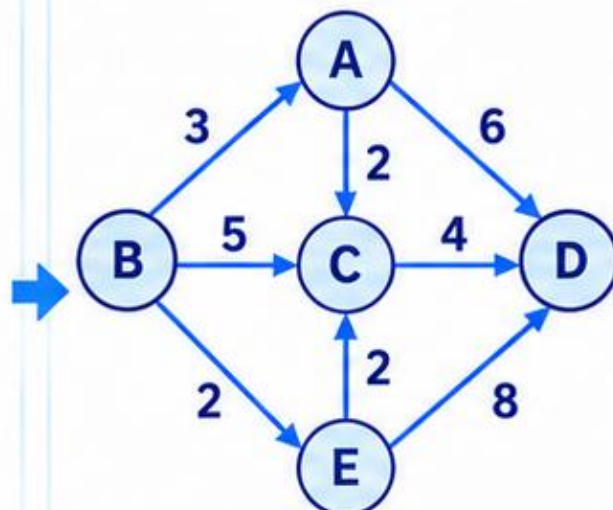
P — Medida de desempenho

- ✓ Chegar ao destino D.
- ✓ Usar apenas ligações permitidas.
- ✓ Ter energia para cada movimento.
- ✓ Terminar com uma reserva mínima de 2 unidades.

Um caminho válido cumpre todas estas condições.

Mapa e movimentos

E — Ambiente



Mapa fornecido: locais, ligações permitidas e energia consumida.

A — Atuadores

Os motores executam os movimentos escolhidos.

u.e. = unidade de energia (dados didáticos).

Critério de escolha

P — Medida de desempenho

Como comparar os caminhos válidos?

Entre os caminhos que cumprem todas as condições, queremos encontrar o que **consome menos energia**.



Minimizar a energia consumida.

O que verificar?

- ? O caminho chega a D?
- ? Todos os movimentos são permitidos e executáveis?
- ? A energia final é suficiente?
- ? Existe outro caminho válido que consuma menos?

i No PEAS, **A** significa **atuadores**. Na procura, as **ações** são os movimentos que esses atuadores executam.



O PEAS descreve a tarefa do agente.



A formulação da procura define os estados e as ações para encontrar um plano.

A seguir: estado inicial, ações, transições, objetivo e custos.

Definir um problema de procura

Formulação do problema

Um problema de procura é descrito por um modelo formal que especifica os estados possíveis, as ações disponíveis, a dinâmica do ambiente, o estado inicial, o teste de objetivo e o custo das ações. Este modelo permite aplicar algoritmos de procura de forma sistemática e rigorosa.

$$P = (S, A, T, s_0, G, c)$$

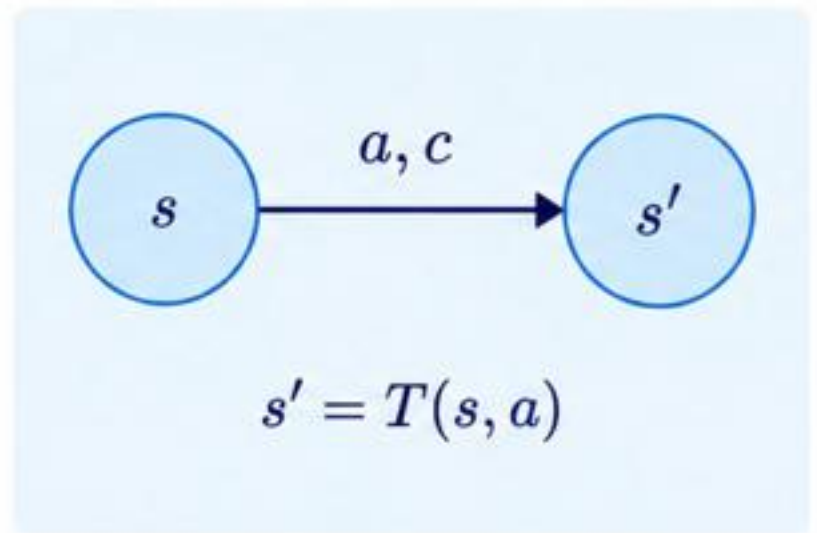
1. Componentes do modelo

O problema é definido por um tuplo com seis elementos.

Símbolo	Descrição
S	Conjunto de estados possíveis.
$A(s)$	Ações permitidas no estado s .
$T(s, a)$	Estado resultante de executar a ação a no estado s .
s_0	Estado inicial.
$G(s)$	Teste de objetivo, que indica se o estado s é um estado de objetivo.
$c(s, a, s')$	Custo de executar a ação a no estado s , resultando no estado s' .

2. Transição entre estados

A execução de uma ação a no estado s produz um novo estado s' e tem um custo c .



Interpretação

O problema de procura consiste em encontrar uma sequência de ações que, a partir do estado inicial s_0 , atinja um estado que satisfaça o teste de objetivo, minimizando o custo total.

3. Hipóteses do modelo

Assumimos um ambiente de procura com as seguintes propriedades:

-  **Modelo conhecido**
O modelo P é completamente conhecido.
-  **Transições determinísticas**
Para cada par (s, a) , o estado resultante é único.
-  **Ambiente estático**
O ambiente não muda com o tempo.
-  **Estados discretos**
O conjunto de estados é finito ou enumerável.
-  **Observação completa**
O estado atual é totalmente observável.

O modelo retém apenas a informação necessária para decidir.

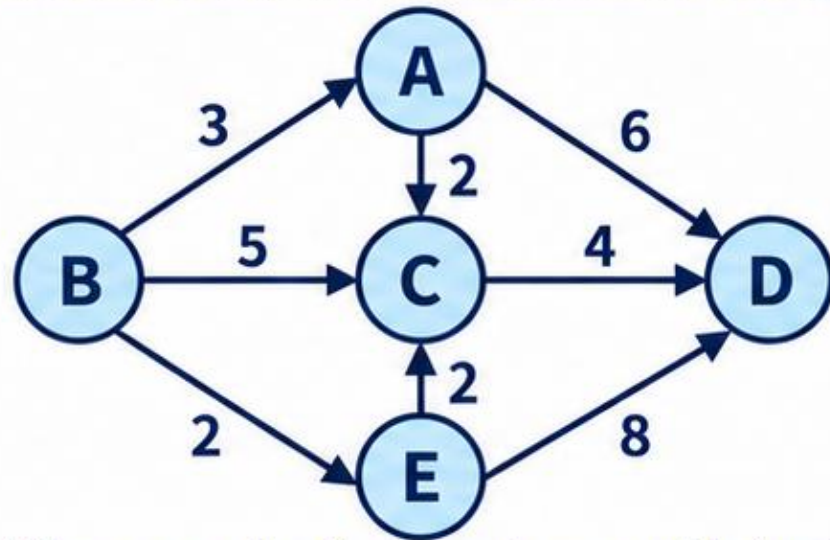
Estado, ação e transição

O grafo mostra as ligações; o estado indica a posição e a energia do robô.

Vamos aplicar o modelo: representar a situação do robô e calcular o que muda quando se desloca.

1. Grafo e estado do robô

O robô só pode deslocar-se pelas ligações indicadas. Cada movimento consome energia.



Números nas ligações: energia consumida (u.e.).

Estado = (posição, energia)
 $s = (p, e)$

Estado inicial: (B, 12)

O robô está em *B* e dispõe de 12 unidades de energia.

Os pontos do grafo representam locais.
O estado inclui também a energia.

2. Ação e transição

A ação $B \rightarrow C$ muda a posição e reduz a energia disponível.



Energia final: $12 - 5 = 7$

A ação é permitida porque:

- Existe a ligação $B \rightarrow C$.
- A energia disponível é suficiente: $12 \geq 5$.

A transição calcula o estado resultante da ação.

$$T((B, 12), B \rightarrow C) = (C, 7)$$

3. A posição não chega

No mesmo local, energias diferentes podem permitir ações diferentes.

Estado	Pode executar $C \rightarrow D$?
(C, 7)	Sim: tem 7 e precisa de 4.
(C, 1)	Não: tem 1 e precisa de 4.



A energia faz parte do estado porque influencia os movimentos que o robô pode executar.

(C, 1) é uma situação hipotética para comparação;
não resulta da transição mostrada.

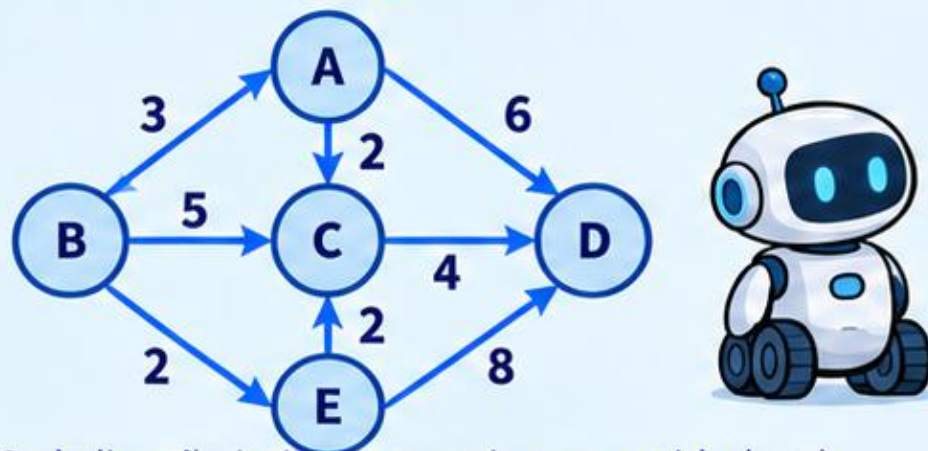
O grafo indica as ligações possíveis. A posição e a energia determinam quais as ações que o robô pode executar.

Objetivo, restrições e custo

Cumprir os requisitos e escolher a solução de menor custo

Já sabemos calcular o resultado de uma ação. Agora vamos verificar se um percurso resolve o problema e comparar o seu custo.

1. O mesmo problema



Cada ligação indica a energia consumida (u.e.).

Estado inicial: (B, 12)

O robô parte de B com 12 unidades de energia.

Só pode usar ligações existentes e executar movimentos para os quais tenha energia suficiente.



2. Quando cumpre o objetivo?

A entrega está concluída quando o robô chega a D e conserva, pelo menos, 2 unidades de energia.

Posição final = D

E

Energia final ≥ 2

$$G(p, e) = (p = D) \wedge (e \geq 2)$$

As duas condições têm de ser cumpridas.

Um estado intermédio pode ser válido sem satisfazer o objetivo.

3. Como comparamos os percursos?

O custo de um percurso é a soma da energia consumida em todas as suas ações.

Custo total = soma dos custos das ações

Energia final = 12 – custo total

Entre os percursos que cumprem os requisitos, procuramos o que consome menos energia.

Importante:

Neste exemplo, o custo é medido em unidades de energia (u.e.).

Dois percursos admissíveis, custos diferentes

Percurso	N.º de ações	Custo total (u.e.)	Energia final (u.e.)	Admissível?	Comparação
B → C → D	2	5 + 4 = 9	12 – 9 = 3	Sim	Cumprir os requisitos
B → E → C → D	3	2 + 2 + 4 = 8	12 – 8 = 4	Sim	Menor custo: solução ótima

Escolhemos B → E → C → D porque consome menos energia (8 < 9) e mantém a reserva exigida.

Uma solução admissível cumpre os requisitos. Uma solução ótima tem o menor custo entre todas as soluções admissíveis.

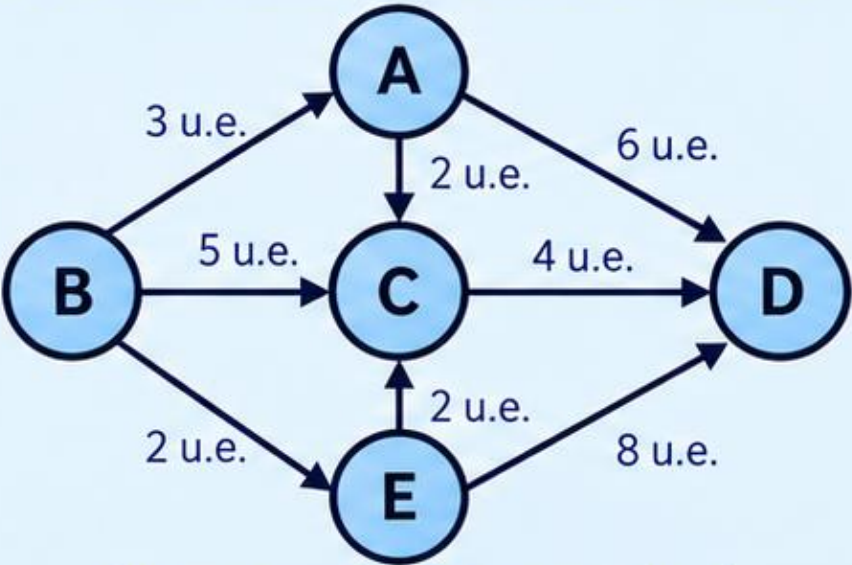
Do estado ao nó de procura

O estado descreve a situação; o nó guarda também como chegámos até ela.

Mantemos o mesmo grafo e o estado inicial (B, 12). Durante a procura, guardamos os estados alcançados e a informação sobre cada percurso.

1. O grafo de locais

O mapa indica os locais, as ligações permitidas e o consumo de energia de cada movimento.



Custos em unidades de energia (u.e.).

Estado inicial: (B, 12)

Posição B; energia disponível 12.

Estado = (posição, energia)

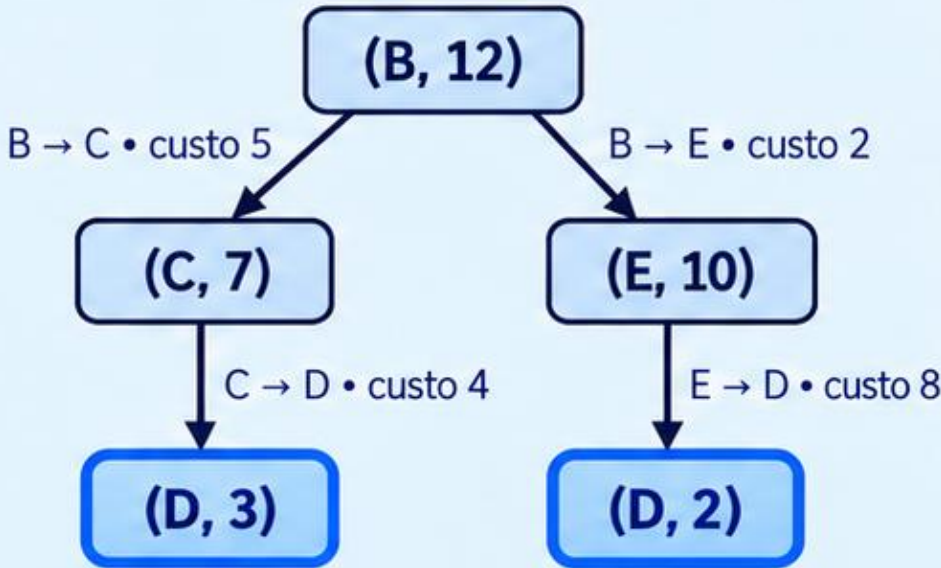
Exemplos: (C, 7), (D, 3) e (D, 2).

Os pontos deste mapa são locais; o estado do robô inclui também a energia.

2. A árvore de procura

Excerto da árvore: dois percursos.

Cada nó representa um estado alcançado por um determinado caminho.



Exemplo detalhado no bloco 3

Mesmo local, estados diferentes

(D, 3) e (D, 2) têm a mesma posição, mas energias diferentes.

3. O que guarda um nó?

Exemplo: nó alcançado pelo percurso B → C → D.

Campo	Valor neste exemplo
Estado	(D, 3)
Pai	Nó com estado (C, 7)
Ação	C → D
Custo acumulado, g	5 + 4 = 9 u.e.
Profundidade, d	2 ações

O custo soma a energia consumida desde o início. A profundidade conta as ações executadas.

nó = (estado, pai, ação, g, d)

Como recuperamos o percurso?

Seguimos os nós pai: (D, 3) → (C, 7) → (B, 12).
Invertendo a ordem, obtemos o percurso: B → C → D.
(Esta sequência é obtida seguindo os nós pai, não são movimentos no grafo.)

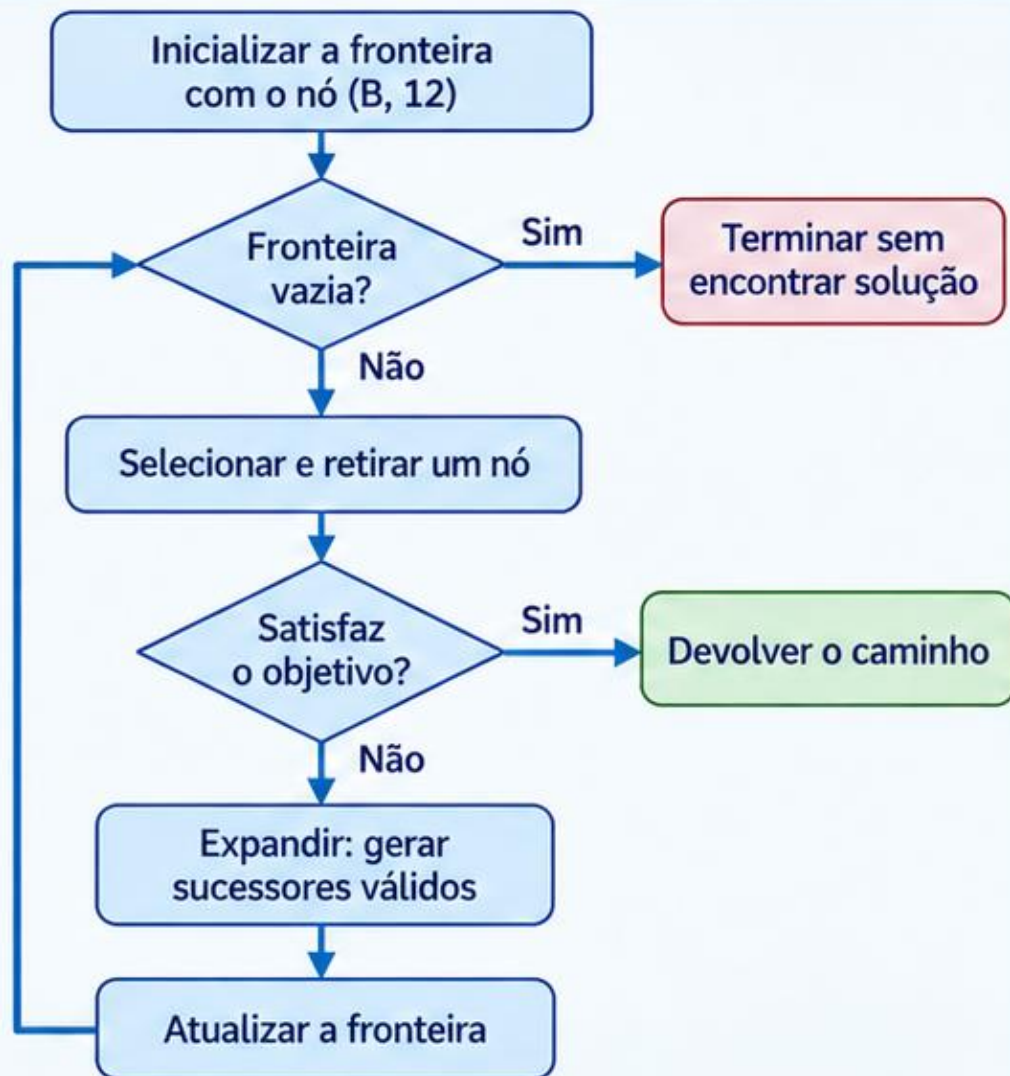
O estado descreve a situação do robô. O nó de procura guarda também o caminho que permitiu alcançá-la.

Fronteira e ciclo geral de procura

Guardar possibilidades, selecionar um nó e gerar os seus sucessores

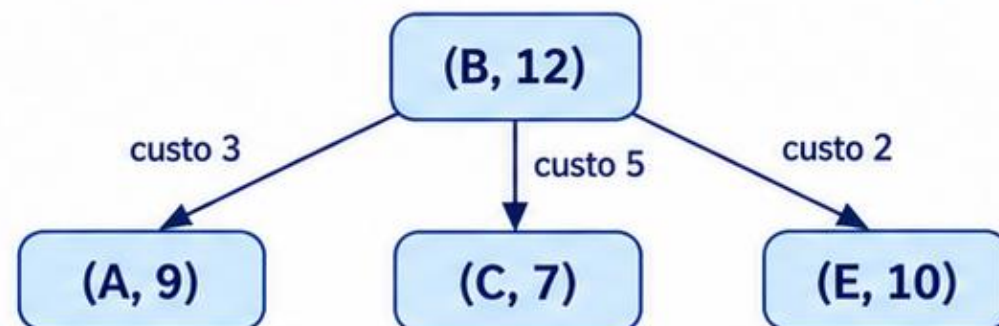
A fronteira contém os nós gerados que aguardam seleção. A estratégia de procura determina qual deles será retirado a seguir.

1. O ciclo de procura



2. Aplicar ao mesmo exemplo

Estado inicial: (B, 12). Objetivo: chegar a D com energia ≥ 2 .



1. Inicializar

Fronteira: nó com estado (B, 12).

2. Retirar (B, 12)

A fronteira fica temporariamente vazia.

3. Testar o objetivo

O robô está em B, não em D. Continuar.

4. Expandir e inserir

A fronteira passa a conter nós com estados (A, 9), (C, 7) e (E, 10).

Mesmo vazia após a retirada, a fronteira só volta a ser testada depois de processarmos o nó.

Qual dos três nós será retirado a seguir?

Depende da estratégia. Ainda não fixámos a ordem.

3. Três operações distintas

Gerar



Criar um nó sucessor.

Retirar



Selecionar um nó e removê-lo da fronteira.

Expandir



Gerar os sucessores válidos do nó selecionado.

Controlar repetições

Um registo de estados alcançados permite reconhecer estados já encontrados.

Dependendo da estratégia, um caminho de menor custo pode exigir atualizar a informação.

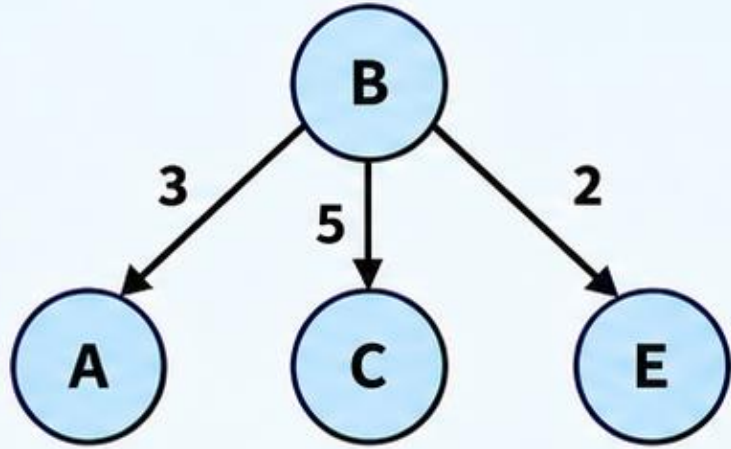
O ciclo mantém-se. BFS, DFS e UCS diferem na regra para escolher o próximo nó da fronteira.

Três regras para selecionar o próximo nó

Procura em largura (BFS), em profundidade (DFS) e de custo uniforme (UCS)

Mesmo mapa, modelo simplificado: para comparar as estratégias, o estado é apenas o local e mantemos os custos.
A reserva de energia será verificada no percurso encontrado: $12 - \text{custo} \geq 2$.

Ligações a partir de B



B é o estado inicial: $g(B) = 0$.

Depois de retirar e expandir B, inserimos os sucessores pela ordem A, C, E.

Sucessor	Custo da ação desde B
A	3
C	5
E	2

Os números nas ligações indicam o custo de cada ação. Como B é o início, $g(B) = 0$.

BFS — Largura

Primeiro a entrar, primeiro a sair

Usa uma fila e seleciona os nós por menor profundidade. Entre estes sucessores, retira o primeiro inserido.



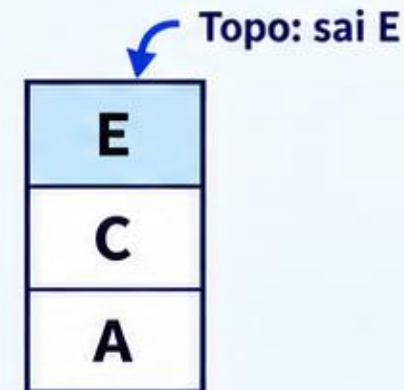
Próximo nó a retirar: A

A foi o primeiro a entrar na fila.

DFS — Profundidade

Último a entrar, primeiro a sair

Usa uma pilha. Com a inserção A, C, E, o nó E fica no topo.



Próximo nó a retirar: E

E foi o último a entrar na pilha. A ordem de inserção influencia o ramo seguido.

UCS — Custo uniforme

Menor custo acumulado primeiro

Retira, de toda a fronteira, o nó com menor custo acumulado desde o início.

$g(\text{filho}) = g(\text{pai}) + \text{custo da ação}$

Nó	Custo acumulado g
E	$0 + 2 = 2$
A	$0 + 3 = 3$
C	$0 + 5 = 5$

Primeiro nó a retirar: E

No passo seguinte

Ao expandir E: C fica com $g = 2 + 2 = 4$ e D com $g = 2 + 8 = 10$.

A: 3	C: 4	D: 10
------	------	-------

A UCS retira A, que já estava na fronteira.

Mesma escolha, razões diferentes

DFS e UCS escolhem E neste momento. A DFS considera a ordem de inserção; a UCS considera o custo acumulado. O controlo de estados repetidos será explicado nos exemplos de cada algoritmo.

A estratégia determina a ordem de retirada da fronteira; não altera os custos das ligações.

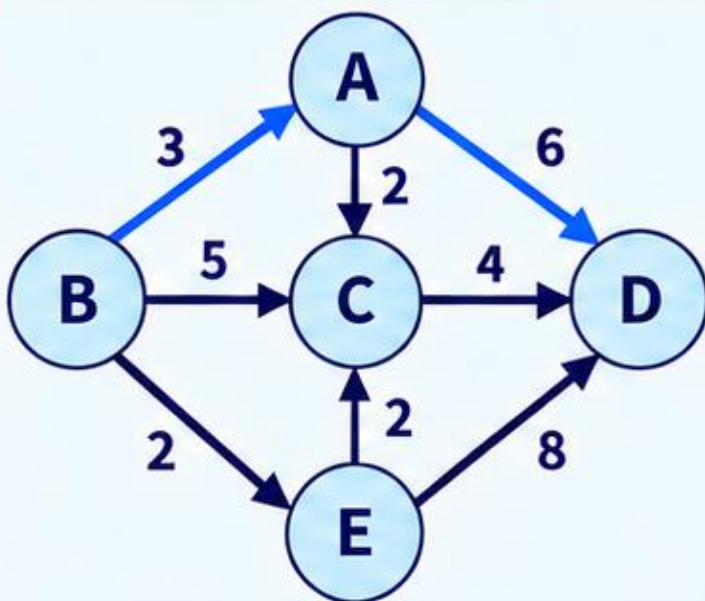
BFS: procura em largura

Explorar por níveis: primeiro 1 ação, depois 2 ações

1. Do grafo à árvore de descoberta

Partimos de **B** e procuramos D. O estado é apenas o local.

Grafo completo



Nas ligações: custo da ação.

Árvore de descoberta da BFS



A, C e E são descobertos ao expandir B. D é descoberto ao expandir A.
C e D não são reinseridos quando voltam a surgir.
A, C e E são retirados antes de D. Os níveis contam ações, não custos.

2. Fila e estados alcançados

Fila FIFO: primeiro a entrar, primeiro a sair.

Depois de expandir B



Retirar e expandir A

- C — já alcançado: não reinserir.
- D — novo: inserir e guardar pai(D) = A.



Retirar da fila não apaga o estado dos alcançados.
D guarda a retirada de C e E.

3. Algoritmo em linguagem simples

1. Criar o nó inicial com $g = 0$, colocá-lo na fila e marcar o seu estado como alcançado.
2. Enquanto a fila não estiver vazia:
 - Retirar o primeiro nó.
 - Se o seu estado satisfaz o objetivo, devolver o caminho.
 - Gerar sucessores por ordem alfabética.
 - Para cada sucessor cujo estado ainda não esteja nos alcançados:
 - Guardar o pai e a ação.
 - Calcular $g(\text{filho}) = g(\text{pai}) + \text{custo da ação}$.
 - Marcar o estado como alcançado.
 - Inserir o nó no fim da fila.
3. Se a fila se esgotar, devolver falha.

g regista o custo do caminho; não altera a ordem FIFO.
A fila guarda nós; os alcançados guardam estados.

O que garante?

- Se existir uma solução e a ramificação for finita, a BFS encontra uma solução com o menor número de ações.
- Com custos iguais e positivos, essa solução também tem o menor custo.

A BFS explora por níveis. A fila garante que os nós menos profundos são retirados primeiro.

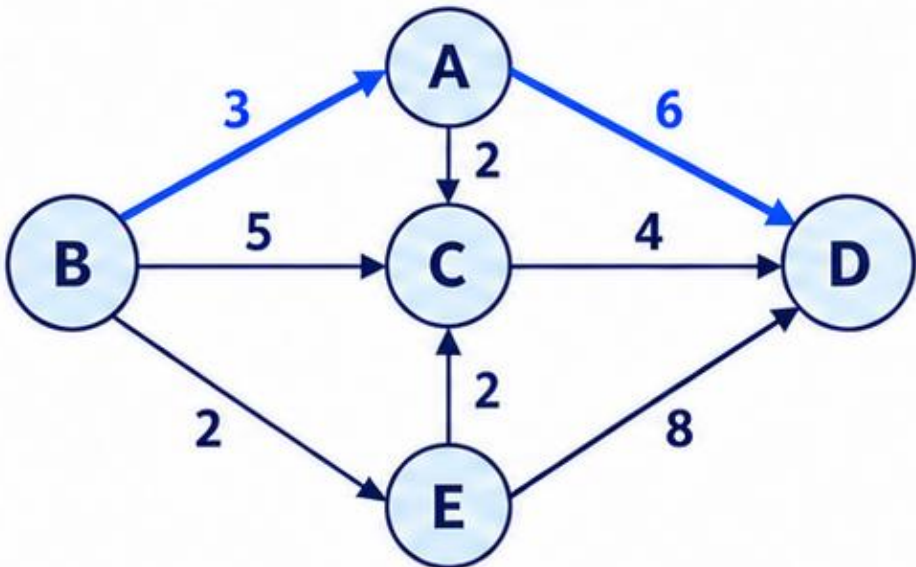
BFS: execução passo a passo

Seguir a fila e compreender o caminho encontrado

Início: B • Objetivo: D • Sucessores por ordem alfabética • Marcar ao inserir • Testar o objetivo ao retirar

1. O grafo completo

O mesmo exemplo: o estado é apenas o local.
Os números representam custos.



Nas setas: custo de cada ação.

O custo das ligações não determina a ordem da fila.
Guardamos o pai de cada estado quando o inserimos pela primeira vez.

2. Evolução da fila

O primeiro nó da fila está à esquerda.

Operação	Fila	d	Explicação
Inicializar	[B]	—	Inserir o nó inicial.
Retirar e expandir B	[A, C, E]	0	Inserir A, C e E.
Retirar e expandir A	[C, E, D]	1	C já foi alcançado. Inserir D, com pai A.
Retirar e expandir C	[E, D]	1	D já foi alcançado; não voltar a inserir.
Retirar e expandir E	[D]	1	C e D já foram alcançados.
Retirar D	[]	2	Objetivo encontrado. D não é expandido.

D entra na fila ao expandir A, mas a procura só termina quando D é retirado.

A, C e E são retirados antes de D porque estão a menor profundidade.

Concluimos o nível 1 antes de retirar D, no nível 2.

3. Resultado

Caminho encontrado
B → A → D

Custo total
3 + 6 = 9

Profundidade
2 ações

Ordem de retirada: B, A, C, E, D.

5 nós retirados

4 nós expandidos

Foram expandidos B, A, C e E.
D foi retirado e reconhecido como objetivo.

Menos ações não significa menor custo

B → A → D: 2 ações, custo 9 — caminho encontrado pela BFS.

B → E → C → D: 3 ações, custo 2 + 2 + 4 = 8 — mais barato, mas mais longo em número de ações.

A BFS dá prioridade ao número de ações, não ao custo acumulado.

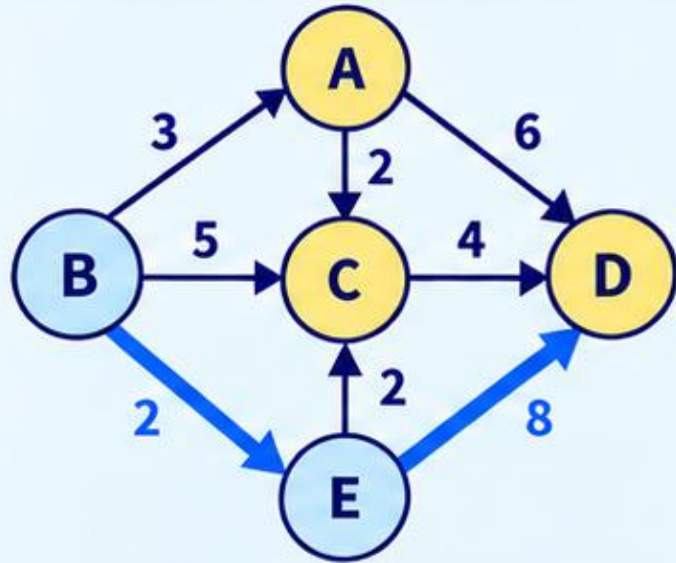
DFS: procura em profundidade

Aprofundar um ramo antes de explorar alternativas

1. Do grafo à árvore de descoberta

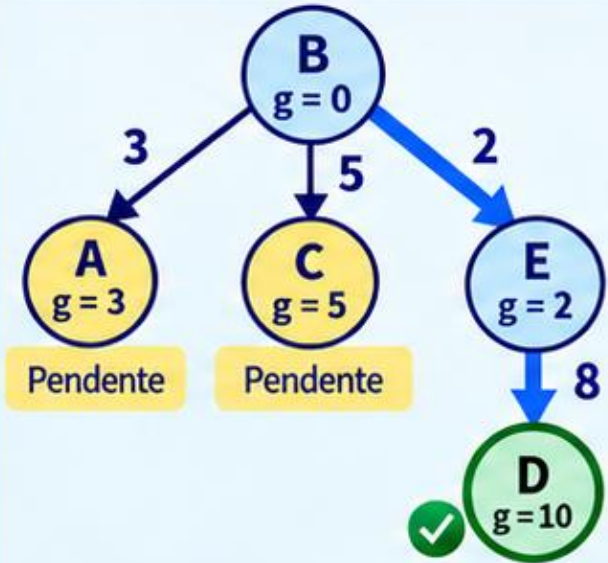
Partimos de B e procuramos D. O estado é apenas o local.

Grafo completo



Números nas ligações = custos das ações.

Árvore de descoberta da DFS



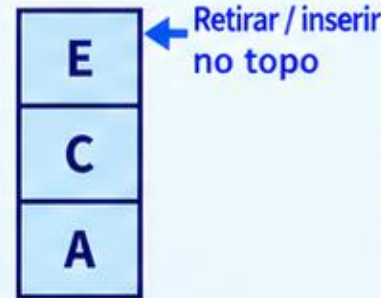
g = custo acumulado desde B.
A DFS não usa g para selecionar nós.
Objetivo encontrado ao retirar D da pilha

Com inserção A, C, E, retiramos E primeiro e seguimos $B \rightarrow E \rightarrow D$. C já foi alcançado ao expandir B; não é reinserido ao expandir E. A e C permanecem na pilha quando retiramos D e terminamos.

2. Pilha e estados alcançados

Pilha LIFO: último a entrar, primeiro a sair.

Depois de expandir B



Alcançados: {B, A, C, E}

Retirar e expandir E

C — já alcançado: não reinserir.
D — novo: guardar $\text{pai}(D) = E$.



Alcançados: {B, A, C, E, D}

Retirar da pilha não apaga o estado dos alcançados. D sai a seguir; A e C ficam pendentes.

3. Algoritmo em linguagem simples

1. Criar o nó inicial com $g = 0$, colocá-lo na pilha e marcar o seu estado como alcançado.
2. Enquanto a pilha não estiver vazia:
 - Retirar o nó do topo.
 - Se o seu estado satisfaz o objetivo, devolver o caminho.
 - Gerar sucessores por ordem alfabética.
 - Para cada sucessor cujo estado ainda não esteja nos alcançados:
 - Guardar o pai e a ação.
 - Calcular $g(\text{filho}) = g(\text{pai}) + \text{custo da ação}$.
 - Marcar o estado como alcançado.
 - Inserir o nó no topo da pilha.
3. Se a pilha se esgotar, devolver falha.

g regista o custo do caminho; não altera a ordem LIFO. A pilha guarda nós; os alcançados guardam estados.

O que garante?

- Num grafo finito, com controlo de estados repetidos, encontra uma solução se existir.
- Num espaço infinito, pode prosseguir indefinidamente num ramo.
- Não garante o menor número de ações nem o menor custo.

A DFS aprofunda um ramo. A pilha guarda as alternativas que poderão ser retomadas.

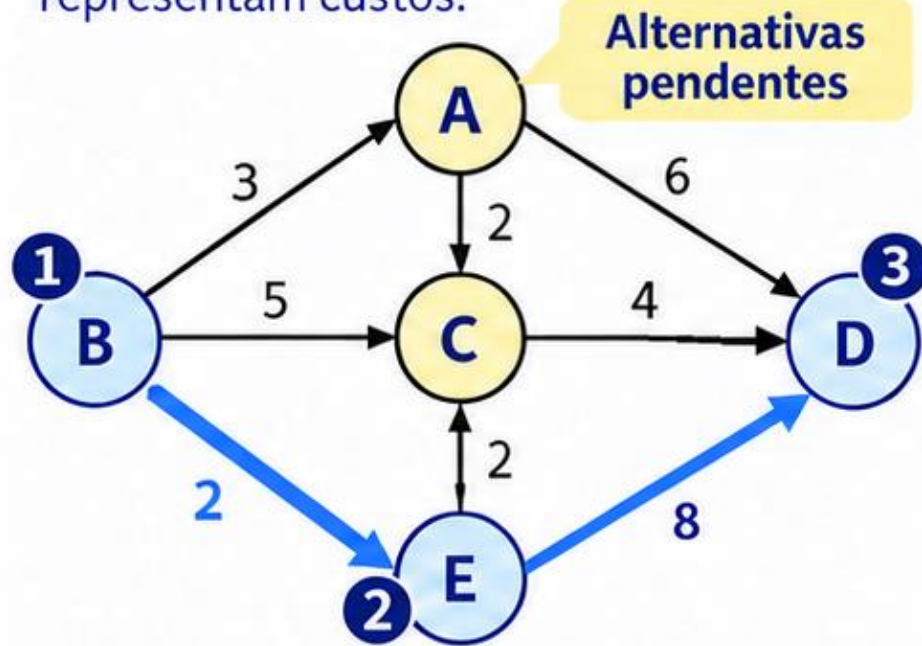
DFS: execução passo a passo

Aprofundar o ramo $B \rightarrow E \rightarrow D$ e manter alternativas na pilha

Início: B • Objetivo: D • Inserção alfabética • Marcar ao inserir • Testar ao retirar

1. O mesmo grafo

Estado: apenas o local. Os números representam custos.



Os custos não determinam a ordem da pilha.

2. Evolução da pilha

Topo à direita: o último elemento é retirado primeiro.

Operação	Pilha	Explicação
Inicializar	[B]	Inserir B.
Retirar e expandir B	[A, C, E]	Inserir A, C e E. E fica no topo.
Retirar e expandir E	[A, C, D]	C já foi alcançado. Inserir D, com pai E.
Retirar D	[A, C]	Objetivo encontrado. D não é expandido.

Pilha no fim:
alternativas não exploradas.

[A] [C] ← Topo

- A DFS segue $B \rightarrow E \rightarrow D$ antes de retomar A ou C. Como D é o objetivo, termina sem precisar de regressar.
- C já tinha sido alcançado ao expandir B; não é reinserido a partir de E.

3. Resultado

Caminho encontrado: $B \rightarrow E \rightarrow D$

Custo total: $2 + 8 = 10$

Profundidade: 2 ações

Ordem de retirada: B, E, D

3 nós retirados

2 nós expandidos

Apenas B e E foram expandidos.
D não foi expandido.

O resultado depende da ordem de inserção

- Com inserção alfabética A, C, E, a DFS segue E primeiro e encontra $B \rightarrow E \rightarrow D$.
- Neste exemplo, encontra uma solução com 2 ações, mas isso não é uma garantia geral.
- Existe um caminho mais barato: $B \rightarrow E \rightarrow C \rightarrow D$, com custo 8.

A primeira solução encontrada pela DFS não é necessariamente a de menor custo.

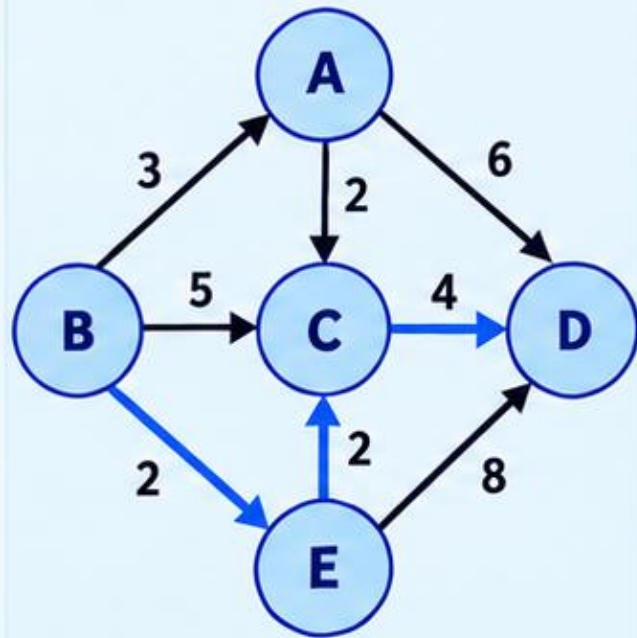
UCS: procura de custo uniforme

Escolher o menor custo acumulado desde o início

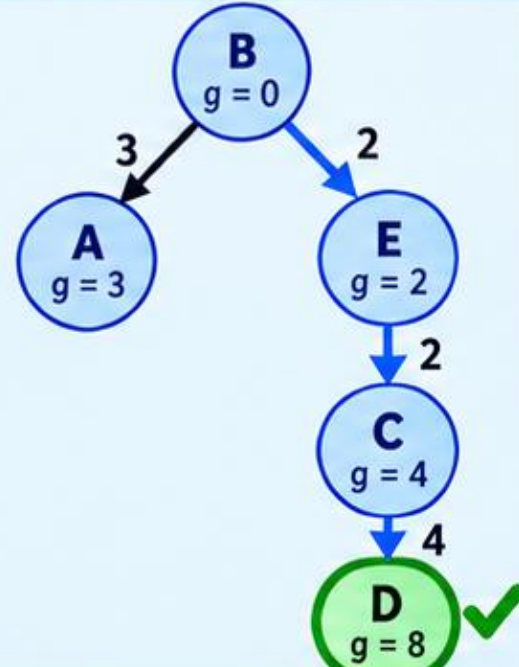
1. Do grafo aos melhores caminhos

Partimos de B e procuramos D. O estado é apenas o local.

Grafo completo



Árvore final de melhores caminhos



Objetivo encontrado ao retirar D.

Nas setas: custo da ação. Nos nós: custo acumulado g .

$$g(\text{filho}) = g(\text{pai}) + \text{custo da ação}$$

Caminho final: $B \rightarrow E \rightarrow C \rightarrow D$. **Custo:** $2 + 2 + 4 = 8$.
A árvore mostra os pais finais, após melhorar os caminhos.

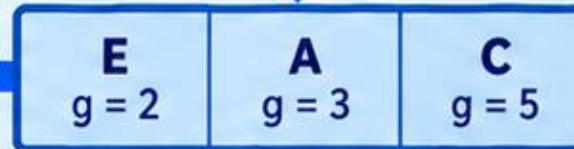
2. Uma fila de prioridade

Retiramos o menor g de toda a fronteira.

Depois de expandir B:

Inserir ou atualizar por prioridade

Retirar
menor g

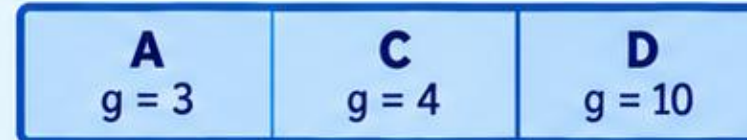


Representação ordenada por g ; não é uma fila FIFO.

Próximo nó a retirar: E

Melhorar caminhos já encontrados

Ao expandir E, C melhora: $5 \rightarrow 2 + 2 = 4$.
O pai de C passa a ser E.



Agora retiramos A, que já estava na fronteira.

A prioridade depende do custo total do caminho,
não apenas da última ação.

3. Algoritmo em linguagem simples

1. Inserir o nó inicial com $g = 0$ e registrar esse custo.
2. Enquanto a fronteira não estiver vazia:
 - Retirar o nó com menor g .
 - Se satisfaz o objetivo, devolver o caminho.
 - Gerar sucessores e calcular os novos custos.
 - Se o estado é novo ou o custo melhora: atualizar custo, pai, ação e prioridade.
3. Se a fronteira se esgotar, devolver falha.

Neste exemplo, substituímos a entrada anterior quando o custo melhora.

Em empate de custo, sai o primeiro inserido.

O que garante?

Num grafo finito, com custos não negativos e atualização correta dos caminhos, encontra uma solução de menor custo, se existir.

Em espaços infinitos, exigem-se condições adicionais:
ramificação finita e custo por ação $\geq \epsilon > 0$.

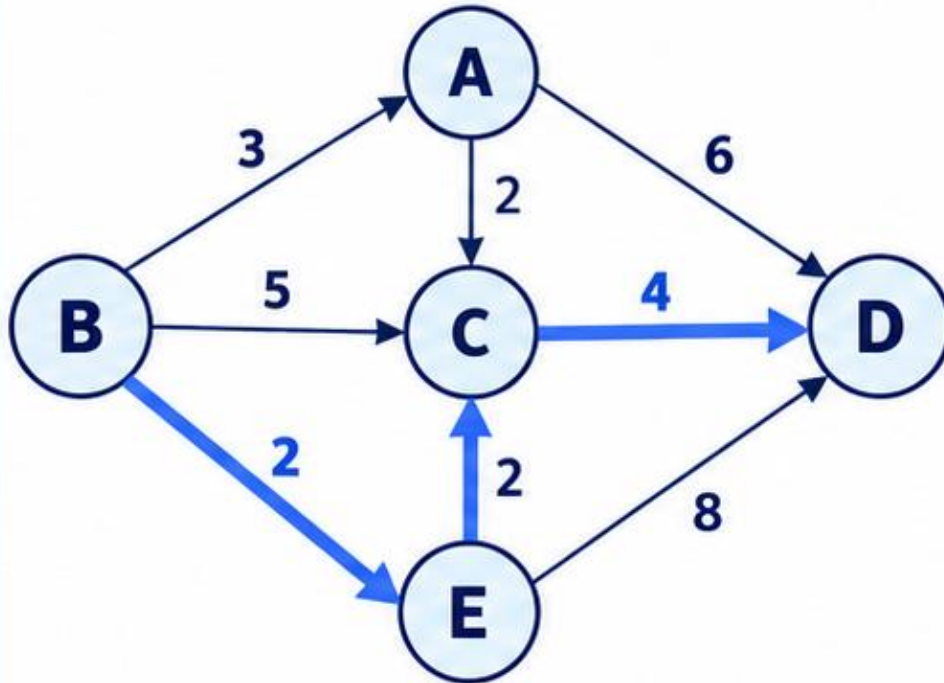
O objetivo é aceite ao retirar o nó de menor custo, não quando é gerado pela primeira vez.

UCS: execução passso o a passo

Atualizar caminhos até retirar o objetivo com o menor custo

Início: B • Objetivo: D • Selecionar menor g • Atualizar se o custo melhorar • Testar ao retirar

1. O mesmo grafo



Estado: apenas o local. g é o custo acumulado desde B.

Um caminho mais barato atualiza o custo, o pai e a prioridade do nó.

2. Evolução da fila de prioridade

Menor g à esquerda. Escrevemos estado:custo.

Operação	Fronteira	Atualizações
Inicializar	[B:0]	Custo inicial 0.
Retirar e expandir B	[E:2, A:3, C:5]	Inserir os três sucessores.
Retirar e expandir E	[A:3, C:4, D:10]	C melhora de 5 para 4; inserir D com 10.
Retirar e expandir A	[C:4, D:9]	Até C: $3+2=5$, não melhora 4. D melhora de 10 para 9.
Retirar e expandir C	[D:8]	D melhora de 9 para 8; pai passa a C.
Retirar D	[]	Objetivo encontrado; não expandir D.

D é gerado com custo 10. A procura continua e melhora esse custo para 9 e depois 8.

A tabela mostra apenas as entradas atuais; as anteriores são substituídas.

3. Resultado

Caminho encontrado:

B → E → C → D

Custo mínimo:

$2 + 2 + 4 = 8$

Profundidade:

3 ações

Ordem de retirada:

B, E, A, C, D

5 nós retirados

4 nós expandidos

Pais finais:

D tem pai C; C tem pai E; E tem pai B.

Porque não terminamos ao gerar D?

O primeiro caminho até D custa 10. Só aceitamos o objetivo quando é retirado como o nó de menor custo da fronteira.

Com os custos não negativos deste grafo, o caminho devolvido tem o menor custo.

A UCS encontra custo 8 com 3 ações. A BFS encontra 2 ações com custo 9: os critérios são diferentes.

Comparar BFS, DFS e UCS

Três estratégias de procura em grafos

BFS (Breadth-First Search), DFS (Depth-First Search) e UCS (Uniform-Cost Search) são algoritmos de procura em grafos que diferem na forma como seleccionam o próximo nó a retirar, o que influencia a solução encontrada, o seu custo e o número de nós explorados.

1. BFS – caminho encontrado (custo 9)



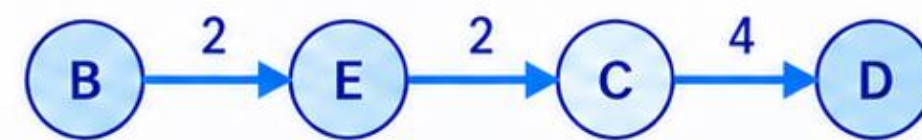
Caminho: $B \rightarrow A \rightarrow D$
Custo total: $3 + 6 = 9$
Energia final: $12 - 9 = 3$

2. DFS – caminho encontrado (custo 10)



Caminho: $B \rightarrow E \rightarrow D$
Custo total: $2 + 8 = 10$
Energia final: $12 - 10 = 2$

3. UCS – caminho encontrado (custo 8)



Caminho: $B \rightarrow E \rightarrow C \rightarrow D$
Custo total: $2 + 2 + 4 = 8$
Energia final: $12 - 8 = 4$

4. Comparação dos resultados

Estratégia	Prioridade	Ordem de retirada	Caminho	Profundidade	Custo	Retirados/expandidos
BFS	FIFO	B, A, C, E, D	$B \rightarrow A \rightarrow D$	2	9	5 / 4
DFS	LIFO	B, E, D	$B \rightarrow E \rightarrow D$	2	10	3 / 2
UCS	menor g	B, E, A, C, D	$B \rightarrow E \rightarrow C \rightarrow D$	3	8	5 / 4

5. Pressupostos da comparação

- Mesmo grafo; estado = local; mesmos custos.
- Geração de sucessores por ordem alfabética.
- Teste de objectivo ao retirar o nó da fronteira (test on pop).
- Na BFS e na DFS, marca-se o estado como alcançado ao inserir.
- Na UCS, atualiza-se a entrada na fronteira quando surge um custo menor.

6. Pseudocódigo genérico

```
fronteira ← {nó inicial}
enquanto fronteira não vazia:
    n ← retirar segundo estratégia
    se G(estado(n)) então:
        devolver caminho(n)
    gerar sucessores(n)
    inserir ou atualizar segundo estratégia)
devolver falha
```

7. Conclusão principal

Menor profundidade não implica menor custo.

A quantidade de nós explorados depende do problema e da ordem de seleção.

Verificar a solução obtida

Uma solução é válida e pode ser ótima, mas devemos sempre verificá-la

Depois de encontrar uma solução candidata, é essencial verificar se todas as condições são satisfeitas (ligações, restrições e objetivo) e se a solução é realmente ótima segundo o critério definido.

1. Ações válidas

Ligações / pré-condições

- Cada ação só pode ser aplicada se as suas pré-condições forem satisfeitas.
- Verificar se todas as ligações do caminho são válidas.

Para cada passagem $X \rightarrow Y$, verificar se existe ligação(X, Y).

2. Objetivo

Estado final

- A solução deve atingir um estado que satisfaça o objetivo.
- Verificar se o estado final obtido cumpre a condição de objetivo.

Verificar se $\text{objectivo}(\text{estado_final})$ é verdadeiro.

3. Restrições

Requisitos

- Todas as restrições do problema devem ser respeitadas (ex.: energia mínima, capacidade).
- Verificar as restrições ao longo do percurso e no estado final.

Verificar se todas as restrições(estado_final) são verdadeiras.

4. Custo

Somatório dos passos

- Calcular o custo total da solução somando o custo de cada ação.
- Comparar o custo com outras soluções, se disponível, para avaliar a optimalidade.

$$\text{custo_total} = \sum \text{custo}(\text{ações})$$

5. Estratégia

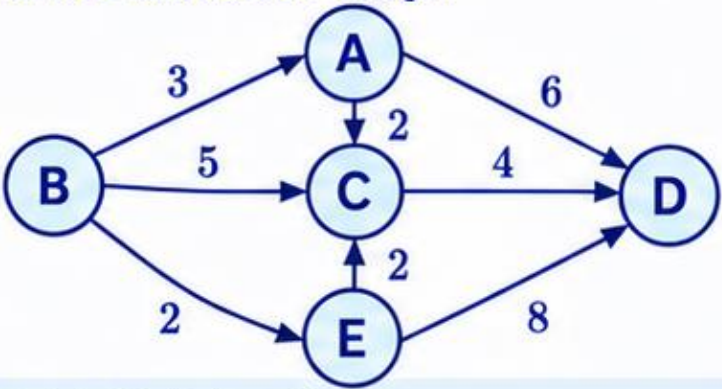
Critério adequado

- Determinar se o algoritmo usa o critério de optimalidade adequado (ex.: menor custo).
- A melhor solução conhecida só é comprovadamente ótima com uma garantia ou verificação adicional.

Confirmar que a estratégia de procura corresponde ao critério pretendido.

Exemplo: robot com energia

Um robot move-se num grafo. Cada ligação tem um custo de energia. A energia inicial em B é 12 e o objetivo é chegar a D com pelo menos 2 unidades de energia.



Energia inicial: 12 (em B)

Objectivo: chegar a D com pelo menos 2 de energia

Cálculo e comparação de soluções

Solução 1: $B \rightarrow C \rightarrow D$

- Custo total: $5 + 4 = 9$
- Energia final: $12 - 9 = 3$
- Posição final: D
- Verificação do objectivo: $G(D, 3) = \text{verdadeiro}$

Solução 2: $B \rightarrow E \rightarrow C \rightarrow D$

- Custo total: $2 + 2 + 4 = 8$
- Energia final: $12 - 8 = 4$
- Posição final: D
- Verificação do objectivo: $G(D, 4) = \text{verdadeiro}$

Ambas as soluções são viáveis, mas $B \rightarrow E \rightarrow C \rightarrow D$ é mais barata.

Verificação detalhada de $B \rightarrow C \rightarrow D$

Critério	Verificação	Resultado
Ligações válidas	Existe ligação(B, C) e ligação(C, D)	Sim
Reserva de energia ≥ 2	Energia final = 3	Sim
Custo total	$5 + 4 = 9$	9
Solução ótima	Existe solução de menor custo ($B \rightarrow E \rightarrow C \rightarrow D$ com custo 8)	Não

Solução válida não é necessariamente ótima.

Perguntas finais

- O modelo corresponde à tarefa?
- O algoritmo garante o critério pedido?

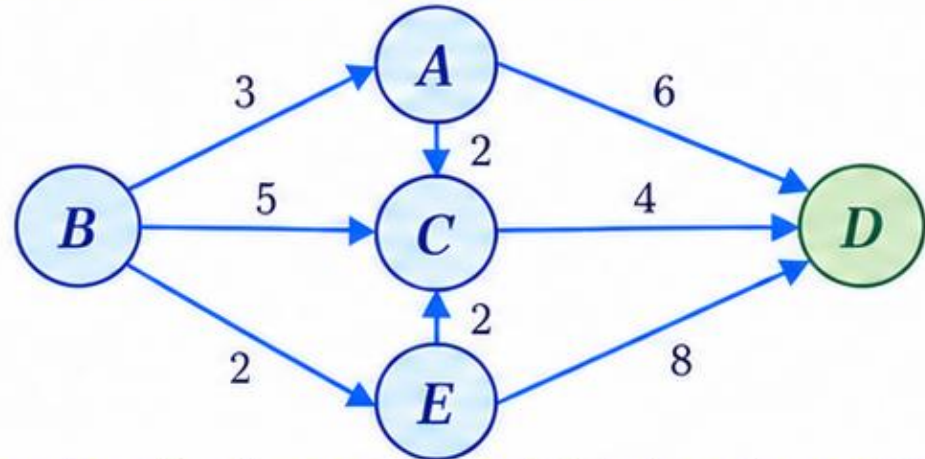
Espaço de estados e alcançabilidade

Análise de quais estados são atingíveis a partir de um estado inicial

A alcançabilidade permite determinar que estados de um espaço de estados podem ser atingidos a partir de um estado inicial, seguindo as transições do sistema. É fundamental em verificação de modelos, planeamento e análise de sistemas.

1. Exemplo de espaço de estados

Mesmo mapa, modelo simplificado: o estado é apenas o local. Estado inicial B; objetivo D.



Os custos não alteram a alcançabilidade neste modelo.

2. Notação e definições

- S Conjunto de estados do modelo.
 - $A(s)$ Conjunto de ações aplicáveis no estado s .
 - $T(s, a)$ Estado sucessor de s após executar a ação a .
 - s_0 Estado inicial.
 - $G(s)$ Predicado que identifica estados objetivo ($G(s) = \text{verdadeiro}$ se s é objetivo).
- Neste exemplo: $S = \{A, B, C, D, E\}$, $s_0 = B$ e $G(s)$ é verdadeiro se $s = D$.

3. Conjunto de estados alcançáveis

Defina R_k como o conjunto de estados alcançáveis em até k ações a partir de s_0 .

$$R_0 = \{s_0\}$$
$$R_{k+1} = R_k \cup \{T(s, a) : s \in R_k, a \in A(s)\}$$
$$R = \bigcup_{k \geq 0} R_k$$

4. Exemplo de cálculo

Aplicando as transições do grafo, obtemos:

k	Conjunto R_k
0	$R_0 = \{B\}$
1	$R_1 = \{B, A, C, E\}$
2	$R_2 = \{B, A, C, E, D\}$
3	$R_3 = R_2 = \{B, A, C, E, D\}$

O conjunto atinge um ponto fixo: $R_{k+1} = R_k$, logo não surgem novos estados alcançáveis.

5. Algoritmo (pseudocódigo)

```
R ← {s0}
enquanto verdadeiro:
    R_novo ← R
    para cada s em R:
        para cada a em A(s):
            s' ← T(s, a)
            R_novo ← R_novo ∪ {s'}

    se R_novo = R:
        retornar R
R ← R_novo
```

6. Observações importantes

- Se não existir nenhum estado $s \in R$ tal que $G(s)$ seja verdadeiro, então não existe solução.
- Num grafo finito, o processo atinge sempre um ponto fixo após um número finito de iterações.
- Neste exemplo, todos os cinco estados são alcançáveis a partir de B: $R = S$.
- Noutros problemas, S pode conter estados não alcançáveis.
- A alcançabilidade permite reduzir o espaço de procura, considerando apenas os estados efetivamente atingíveis.

Tempo e memória da procura

Análise de desempenho de algoritmos de procura

O tempo de execução e a memória necessária dependem do factor de ramificação, da profundidade da solução e da estratégia de procura. Esta análise permite prever os recursos necessários e comparar diferentes algoritmos.

1. Parâmetros e notação

$b \geq 2$	factor de ramificação máximo (número máximo de sucessores por estado).
d	profundidade da solução mais rasa (<i>shallowest goal depth</i>).
m	profundidade máxima da procura (assumida finita).

2. Complexidade na procura em árvore

Complexidade temporal e de memória da fronteira para os algoritmos básicos de procura em árvore.

Árvore de procura	Tempo	Memória da fronteira
BFS (teste ao retirar)	$O(b^{d+1})$	$O(b^{d+1})$
DFS (m finito)	$O(b^m)$	$O(bm)$

3. BFS pode gerar o nível $d + 1$

Com teste de objetivo ao retirar, o BFS pode expandir outros nós do nível d antes de retirar o objetivo. Assim, podem existir nós gerados no nível $d + 1$.

Esta possibilidade é geral; no nosso grafo, a BFS não gera novos estados no nível 3.

Objetivo ao retirar →
podem existir nós gerados no nível $d + 1$.

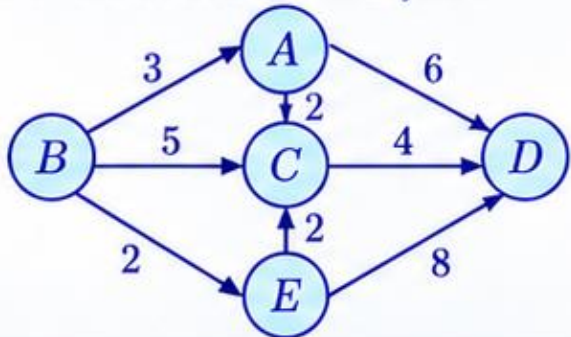
4. Exemplo teórico de crescimento ($b = 3$)

Árvore completa com 3 sucessores por nó; não é a contagem do nosso grafo.

Nível (profundidade)	Número de nós	Cálculo
0	1	$3^0 = 1$
1	3	$3^1 = 3$
2	9	$3^2 = 9$
3	27	$3^3 = 27$

5. Procura em grafos e memória global

Na procura em grafos, mantém-se um conjunto global de estados já alcançados (*reached*). Este conjunto pode crescer até $|V|$, o número de estados alcançáveis.



Memória adicional:
 $O(|V|)$
Mesmo mapa:
estado = local; $|V| = 5$.

6. Pseudocódigo (BFS em grafos)

```
função BFS(inicial)
  fronteira ← [inicial]
  alcançados ← {inicial}
  enquanto fronteira não está vazia
    n ← retirar_primeiro(fronteira) # FIFO
    se objetivo(n) então
      retorna n
    para cada sucessor em sucessores(n)
      se sucessor não está em alcançados então
        alcançados.adicionar(sucessor)
        fronteira.inserir_no_fim(sucessor) # no fim
    alcançados ← alcançados ∪ {sucessor}
  retorna falha
```

7. Observações importantes

- Na procura em grafos, a memória global pode ser $O(|V|)$, mesmo que a pilha da DFS seja pequena.
- A memória da fronteira **não** é a memória total, pois é necessário incluir a memória global de estados alcançados.
- O tempo e a memória crescem rapidamente com b e d , mesmo para valores moderados.

Registrar nós e reconstruir caminhos

Representar a informação para recuperar a solução

Em algoritmos de procura em grafos ou em espaços de estados, cada nó guarda informação que permite reconstruir o caminho até ao estado inicial. Depois de encontrar um estado objetivo, seguimos os ponteiros de pai até à raiz e obtemos a sequência de estados e o respetivo custo.

1. Informação guardada em cada nó

Cada nó n contém os seguintes campos:

Campo	Significado
estado $s(n)$	configuração do estado
pai(n)	nó anterior
ação(n)	ação que gerou n
$g(n)$	custo acumulado desde o estado inicial
$d(n)$	profundidade (número de ações)

2. Atualização dos campos

Quando se gera um nó filho a partir de um nó pai:

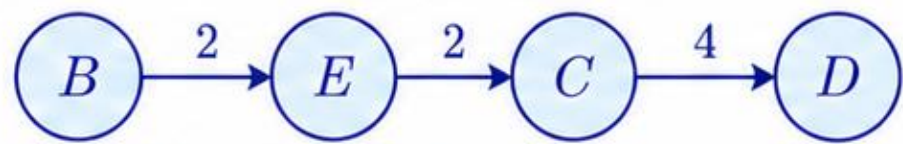
$$g(\text{filho}) = g(\text{pai}) + c(s, a, s')$$
$$d(\text{filho}) = d(\text{pai}) + 1$$

onde:

- $c(s, a, s')$ é o custo da ação a que leva de s para s' ;
- $g(n)$ é o custo acumulado desde o estado inicial;
- $d(n)$ é a profundidade do nó.

3. Percurso encontrado pela UCS no mesmo grafo

Mostramos apenas as ligações do percurso; estado = local.



4. Registos finais do percurso

A tabela mostra os valores dos campos principais:

Nó	$g(n)$	$d(n)$	pai(n)
B	0	0	nulo
E	2	1	B
C	4	2	E
D	8	3	C

Na UCS, estes registos são atualizados quando se encontra um caminho de menor custo.

5. Reconstrução do caminho

Seguindo os ponteiros de pai a partir de D :



Invertendo a sequência, obtemos o caminho:

$$B \rightarrow E \rightarrow C \rightarrow D \quad (\text{custo total} = 8)$$

6. Algoritmo de reconstrução do caminho

Dado um nó objetivo n , o seguinte algoritmo devolve a lista de estados do caminho desde o estado inicial até n :

```
caminho ← []
enquanto n ≠ nulo:
    acrescentar estado(n) a caminho
    n ← pai(n)
devolver inverter(caminho)
```

7. Observações importantes

- O campo $pai(n)$ permite reconstruir o caminho sem guardar toda a sequência de ações durante a procura.
- O custo total do caminho é dado por $g(n)$ no nó objetivo.
- A profundidade $d(n)$ corresponde ao número de ações desde o estado inicial.
- O algoritmo recolhe os estados de D para B e depois inverte a sequência para obter o caminho na ordem correta.
- Esta técnica aplica-se a várias estratégias de procura, como a procura em largura, a procura em profundidade e a procura de custo uniforme.

Garantias e escolha da estratégia

Como as propriedades do problema influenciam a escolha do algoritmo

Diferentes estratégias de procura têm garantias distintas de completude, optimalidade e custo, dependendo das características do espaço de estados e dos custos das ações. Esta síntese ajuda a escolher o algoritmo mais adequado para cada situação.

1. Espaço de estados finito (com repetição de estados controlada)

Algoritmo	Completude	Menor profundidade	Custo ótimo	Condições / Observações
BFS	Sim	Sim	Sim (se custos iguais e positivos)	FIFO e controlo de repetidos.
DFS	Sim	Não	Não	Pode encontrar uma solução mais profunda e não garante o menor custo.
UCS	Sim	Não	Sim	Com custos não negativos e atualização correta de estados na fronteira (mantém o menor custo).

2. Espaços potencialmente infinitos

Algoritmo	Completude	Menor profundidade	Custo ótimo	Condições / Observações
BFS	Sim	Sim	Sim (se custos iguais e positivos)	Se a ramificação é finita e existe uma solução a profundidade finita.
DFS	Não	Não	Não	Pode não terminar (ex.: seguir um ramo infinito).
UCS	Sim	Não	Sim	Se a ramificação é finita, existe uma solução de custo finito e o custo de cada ação é $\geq \epsilon > 0$.

ϵ define um limite inferior positivo para o custo de cada ação.

3. Escolha da estratégia

Menor número de ações
Usar **BFS**.



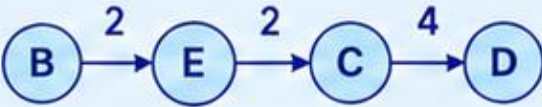
Neste grafo: 2 ações, custo 9.
Só garante menor custo quando os custos são iguais e positivos.

Explorar em profundidade
Usar **DFS**.



Neste grafo: 2 ações, custo 10.
O resultado depende da ordem de inserção; não garante menor profundidade nem menor custo.

Minimizar custos variáveis não negativos
Usar **UCS**.



Retira sempre o nó com menor custo acumulado. Garante uma solução ótima com custos não negativos.

4. Questão final

Porque pode o UCS escolher mais ações e ainda assim uma solução melhor?

O UCS minimiza o custo total, não o número de ações. Assim, pode encontrar uma solução com mais ações mas de menor custo.

Solução encontrada pelo UCS

Custo total = 8
Número de ações = 3

Solução encontrada pelo BFS

Custo total = 9
Número de ações = 2

Neste exemplo, o UCS escolhe uma solução com mais ações (3 em vez de 2) mas com custo total menor (8 em vez de 9).

Não = sem garantia geral; pode acontecer num exemplo.